МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере информационных технологий Образовательного центра Института №8

Отчет по лабораторной работе

по дисциплине «Операционные системы»

на тему: **«Диагностика системных вызовов»**

Выполнил:

студент гр. М8О-205БВ-25

Батурин Эдуард Дмитриевич

Проверил: Филимонов Н. С.

Оценка:

Дата:

Москва 2026

Цель работы: приобретение практических навыков диагностики работы программного обеспечения.

Задание: Необходимо выбрать стороннее ПО (возможные варианты ПО указаны ниже) и составить профиль его системных вызовов. Профиль составляется при помощи средств диагностики, указанных ниже в зависимости от выбранной операционной системы. Профиль должен содержать группы найденных системных вызовов и указания функционала ПО для чего они вызывались.

Ход работы

Для диагностики я выбрал программу “ripgrep”. Диагностика будет проводиться с помощью “strace” на Ubuntu 24.04. В заранее заготовленной директории я ввёл “strace rg -z "main"”. Первой строчкой вывелось execve("/usr/bin/rg", ["rg", "-z", "main"], 0x7ffe8261b8d0 /* 51 vars */) = 0. Это системный вызов, который загружает новую программу в адресное пространство текущего процесса и начинает её выполнение. 0x7ffe8261b8d0 - адрес массива переменных окружения. Далее с помощью brk(NULL) = 0x589a38ca0000 узнает размер кучи. С помощью mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7ff0c595b000 выделяется виртуальная память. Проверяется библиотека, которая отсутствует access("/etc/ld.so.preload", R_OK)   = -1 ENOENT (No such file or directory). Далее удалось уже открыть файл с информацией о расположении библиотек openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3. Далее fstat(3, {st_mode=S_IFREG|0644, st_size=46247, ...}) = 0 получает данные о нем, что он простой, права доступа, размер. Идет другой mmap(NULL, 46247, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7ff0c594f000 но он связан со чтением дескриптора 3, что есть /etc/ld.so.cache. close(3)   Дескриптор файла закрывается, но mmap остаётся. Затем с помощью блока

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 832) = 832

fstat(3, {st_mode=S_IFREG|0644, st_size=625344, ...}) = 0

mmap(NULL, 627472, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7ff0c58b5000

mmap(0x7ff0c58b7000, 450560, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x7ff0c58b7000

mmap(0x7ff0c5925000, 163840, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x70000) = 0x7ff0c5925000

mmap(0x7ff0c594d000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x97000) = 0x7ff0c594d000

close(3) = 0 открывается библиотека libpcre2-8.so.0, читается заголовок 832 байта, проверяется его размер, тип, права, загружаются разные области этой библиотеки в разные области памяти, дескриптор закрывается. Затем идет аналогичный блок openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"..., 832) = 832

fstat(3, {st_mode=S_IFREG|0644, st_size=183024, ...}) = 0

mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7ff0c5887000

mmap(0x7ff0c588b000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x4000) = 0x7ff0c588b000

mmap(0x7ff0c58af000, 16384, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) =
0x7ff0c58af000
mmap(0x7ff0c58b3000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2b000) =
0x7ff0c58b3000
close(3) = 0, однако теперь он для libgcc_s.so.1. Затем
загружается следующая библиотека в блоке openat(AT_FDCWD,
"/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"...,
832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...,
784, 64) = 784
mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3,
0) = 0x7ff0c5600000
mmap(0x7ff0c5628000, 1609728, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) =
0x7ff0c5628000
mmap(0x7ff0c57b1000, 323584, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b1000) =
0x7ff0c57b1000
mmap(0x7ff0c5800000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ff000) =
0x7ff0c5800000
mmap(0x7ff0c5806000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7ff0c5806000

close(3) = 0. Тут загружается libc.so.6. Тут тоже аналогично с предыдущими библиотеками, однако есть еще pread64, который читает данные и сохраняет их в отдельный буфер. Цель mmap при загрузке библиотек - отображение файла в виртуальной памяти, после чего программа может обращаться к его содержимому через обычные указатели. Далее идет блок mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7ff0c5885000
arch_prctl(ARCH_SET_FS, 0x7ff0c5885c40) = 0
set_tid_address(0x7ff0c5885f10) = 24647
set_robust_list(0x7ff0c5885f20, 24) = 0
rseq(0x7ff0c5886560, 0x20, 0, 0x53053053) = 0
mprotect(0x7ff0c5800000, 16384, PROT_READ) = 0
mprotect(0x7ff0c58b3000, 4096, PROT_READ) = 0
mprotect(0x7ff0c594d000, 4096, PROT_READ) = 0
mprotect(0x5899fa465000, 319488, PROT_READ) = 0
mprotect(0x7ff0c5993000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7ff0c594f000, 46247) = 0
poll([{fd=0, events=0}, {fd=1, events=0}, {fd=2, events=0}], 3, 0) = 0 (Timeout)
rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[PIPE],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7ff0c5645330},
{sa_handler=SIG_DFL, sa_mask=[], sa_flags=0}, 8) = 0
rt_sigaction(SIGSEGV, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0},
8) = 0
rt_sigaction(SIGSEGV, {sa_handler=0x5899fa2fdc70, sa_mask=[],
sa_flags=SA_RESTORER|SA_ONSTACK|SA_SIGINFO,
sa_restorer=0x7ff0c5645330}, NULL, 8) = 0
rt_sigaction(SIGBUS, NULL, {sa_handler=SIG_DFL, sa_mask=[], sa_flags=0},
8) = 0

rt_sigaction(SIGBUS, {sa_handler=0x5899fa2fdc70, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA_SIGINFO, sa_restorer=0x7ff0c5645330}, NULL, 8) = 0
sigaltstack(NULL, {ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=0}) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7ff0c5958000
mprotect(0x7ff0c5958000, 4096, PROT_NONE) = 0
sigaltstack({ss_sp=0x7ff0c5959000, ss_flags=0, ss_size=8192}, NULL) = 0
getrandom("\x9e\xd5\x58\x1c\xa3\xef\x37\xe1", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x589a38ca0000
brk(0x589a38cc1000) = 0x589a38cc1000. В нем есть новые виды системных вызовов. mmap выделяет память в ОЗУ. arch_prctl устанавливает регистр FS. Затем set_tid_address записывает айди текущего потока, это нужно, чтобы в будущем использовать futex и блокироваться до изменения этого айди. set_robust_list(0x7ff0c5885f20, 24) = 0 продолжает инициализацию мьютексов, если поток умрет и не вызовет unlock сам, то мьютекс все равно разблокируется. rseq нужен, чтобы программа могла гарантированно выполнить кусок кода на определенном потоке, если ядро распорядится иначе, то будет вызван специальный обработчик. Далее много mprotect нужно для того, чтобы запретить писать в память библиотек и т.п. prlimit64 снова спрашивает размер стека. munmap отменяет mmap (/etc/ld.so.cache). poll([{fd=0, events=0}, {fd=1, events=0}, {fd=2, events=0}], 3, 0) = 0 (Timeout) проверяет, что 3 стандартных файловых дескриптора открыты. Это делает стандартная библиотека Rust. Затем множество rt_sigaction, где настраиваются разные функции обработки сигналов, например, SIGPIPE игнорируется, чтобы не завершать процесс сразу, как пайп закончился. Субблок sigaltstack(NULL, {ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=0}) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7ff0c5958000

mprotect(0x7ff0c5958000, 4096, PROT_NONE) = 0
sigaltstack({ss_sp=0x7ff0c5959000, ss_flags=0, ss_size=8192}, NULL) = 0
нужен для инициализации альтернативного стека для обработки случая, если основной стек закончился. Затем
getrandom("\x9e\xd5\x58\x1c\xa3\xef\x37\xe1", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x589a38ca0000
brk(0x589a38cc1000) = 0x589a38cc1000 узнаёт случайные 8 байт (возможно для seed псевдослучайного генератора чисел), а также проверяет и увеличивает heap. Затем идёт блок openat(AT_FDCWD, "/proc/self/maps", O_RDONLY|O_CLOEXEC) = 3
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "5899f9fb0000-5899fa02f000 r--p 0"..., 1024) = 1024
read(3, "86_64-linux-gnu/libc.so.6\n7ff0c5"..., 1024) = 1024
read(3, "gnu/libpcre2-8.so.0.11.2\n7ff0c59"..., 1024) = 1024
read(3, " [stack]\n7ffff8bc3000"..., 1024) = 180
close(3). В приведенном выше открывается /proc/self/maps, а также проверяется лимит стека и читаются ранее за’mmap’ленные куски памяти, закрывается дескриптор. Затем идёт блок sched_getaffinity(24647, 32, [0 1 2 3 4 5 6 7 8 9 10 11]) = 8
getrandom("\x11\xfc\xfb\x9e\xdf\xb3\xa0\xaf\x50\x1e\x33\xb0\x74\xe3\x2f\x60", 16, GRND_INSECURE) = 16
ioctl(1, TCGETS, {c_iflag=ICRNL,
c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|OPOST|ONLCR,
c_cflag=B38400|CS8|CREAD,
c_lflag=ISIG|ICANON|ECHO|ECHOE|ECHOK|IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
getcwd("/home/trard/dev/c", 512) = 18

ioctl(0, TCGETS, {c_iflag=ICRNL,
c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|OPOST|ONLCR,
c_cflag=B38400|CS8|CREAD,
c_lflag=ISIG|ICANON|ECHO|ECHOE|ECHOK|IEXTEN|ECHOCTL|ECHOKE,
...}) = 0
uname({sysname="Linux", nodename="DESKTOP-DM4FNVR", ...}) = 0
brk(0x589a38ce2000) = 0x589a38ce2000, где продолжается инициализации программы. Проверяется на каких CPU можно исполняться для потока, getrandom снова получает случайные байты. Затем узнаются настройки моего терминала (например ISIG — специальные комбинации клавиш могут генерировать сигналы, например Ctrl+C → SIGINT или ONCLR — при выводе преобразовывать \n в последовательность \r\n), имя системы, текущую рабочую директорию, версию ядра и т.п. Увеличивается стек. Затем идет блок openat(AT_FDCWD, "/proc/self/cgroup",
O_RDONLY|O_CLOEXEC) = 3
statx(3, "", AT_STATX_SYNC_AS_STAT|AT_EMPTY_PATH, STATX_ALL,
{stx_mask=STATX_BASIC_STATS|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFREG|0444, stx_size=0, ...}) = 0
lseek(3, 0, SEEK_CUR) = 0
read(3, "0::/init.scope\n", 128) = 15
read(3, "", 113) = 0
close(3) = 0
statx(AT_FDCWD, "/sys/fs/cgroup/init.scope/cgroup.controllers",
AT_STATX_SYNC_AS_STAT, STATX_ALL,
{stx_mask=STATX_BASIC_STATS|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFREG|0444, stx_size=0, ...}) = 0
openat(AT_FDCWD, "/sys/fs/cgroup/init.scope/cpu.max",
O_RDONLY|O_CLOEXEC) = 3

statx(3, "", AT_STATX_SYNC_AS_STAT|AT_EMPTY_PATH, STATX_ALL,
{stx_mask=STATX_BASIC_STATS|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFREG|0644, stx_size=0, ...}) = 0
lseek(3, 0, SEEK_CUR)   = 0
read(3, "max 100000\n", 20)   = 11
read(3, "", 9)   = 0
close(3)   = 0. С помощью этого блока ripgrep узнает сколько ему разрешено использовать cpu согласно ограничениям cgroup, чтобы не исчерпать квоту. Далее идет блок statx(AT_FDCWD, "./",
AT_STATX_SYNC_AS_STAT, STATX_ALL,
{stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFDIR|0755, stx_size=4096, ...}) = 0
statx(1, "", AT_STATX_SYNC_AS_STAT|AT_EMPTY_PATH, STATX_ALL,
{stx_mask=STATX_BASIC_STATS|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFCHR|0620, stx_size=0, ...}) = 0
statx(1, "", AT_STATX_SYNC_AS_STAT|AT_EMPTY_PATH, STATX_ALL,
{stx_mask=STATX_BASIC_STATS|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFCHR|0620, stx_size=0, ...}) = 0
openat(AT_FDCWD, "/home/trard/.gitconfig", O_RDONLY|O_CLOEXEC) = 3
statx(3, "", AT_STATX_SYNC_AS_STAT|AT_EMPTY_PATH, STATX_ALL,
{stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFREG|0644, stx_size=342, ...}) = 0
lseek(3, 0, SEEK_CUR)   = 0
read(3, "[user]\n\temail = trardone@gmail.c"..., 342) = 342
read(3, "", 32)   = 0
close(3)   = 0. В нём узнается информация о текущей директории (права и т.п.), а также проверяется stdin (это S_IFCHR — символьное устройство). Читается .gitconfig. Затем снова растет куча
brk(0x589a38d04000)   = 0x589a38d04000. Проверяется отсутствие

файлов openat(AT_FDCWD, "/home/trard/.config/git/config", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory) statx(AT_FDCWD, "/home/trard/.config/git/ignore", AT_STATX_SYNC_AS_STAT, STATX_ALL, 0x7ffff8b11e40) = -1 ENOENT (No such file or directory). А далее есть интересная часть statx(0, NULL, AT_STATX_SYNC_AS_STAT, STATX_ALL, NULL) = -1 EFAULT (Bad address) statx(AT_FDCWD, "./", AT_STATX_SYNC_AS_STAT, STATX_ALL, {stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFDIR|0755, stx_size=4096, ...}) = 0. Снова проверяется текущая директория, а также Rust проверяет работает ли вообще системный вызов statx, вызывая его с NULL, после того как не нашелся файл /home/trard/.config/git/ignore. Затем снова растет heap brk(0x589a38d26000) = 0x589a38d26000. После этого идёт основная часть работы программы уже rt_sigaction(SIGRT_1, {sa_handler=0x7ff0c5699610, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7ff0c5645330}, NULL, 8) = 0 rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0 mmap(NULL, 2101248, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7ff0c53ff000 mprotect(0x7ff0c5400000, 2097152, PROT_READ|PROT_WRITE) = 0 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0. Тут устанавливается обработчик SIGRT_1 - сигнала реального времени 1, это нужно для общения между потоками. rt_sigprocmask разблокирует сигнал, то есть как бы разрешает его получение. Затем mmap выделяет озу для стека нового потока и защищается страница, чтобы не выйти за границы стека случайно. rt_sigprocmask блокирует сигналы, т.к. новый поток наследует обрабатываемые сигналы и нужно, чтобы он во время инициализации не обрабатывал их. Затем непосредственно создается сам поток и разблокируются сигналы clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CL

ONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SET
TID|CLONE_CHILD_CLEARTID, child_tid=0x7ff0c55ff990,
parent_tid=0x7ff0c55ff990, exit_signal=0, stack=0x7ff0c53ff000,
stack_size=0x1ffec0, tls=0x7ff0c55ff6c0} => {parent_tid=[24648]}, 88) = 24648
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0. Из важного тут есть
CLONE_CHILD_CLEARTID, что очищает TID при завершении потока, чтобы
потом можно было использовать futex по TID. Затем результаты strace
мешаются с stdout ripgrep. Затем идет многократное создание потоков
mmap(NULL, 2101248, PROT_NONE,
MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7ff0c49fa000
mprotect(0x7ff0c49fb000, 2097152, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CL
ONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SET
TID|CLONE_CHILD_CLEARTID, child_tid=0x7ff0c4bfa990,
parent_tid=0x7ff0c4bfa990, exit_signal=0, stack=0x7ff0c49fa000,
stack_size=0x1ffec0, tls=0x7ff0c4bfa6c0} => {parent_tid=[24653]}, 88) = 24653
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0. Их создается 12 штук, чтобы
можно было максимально эффективно использовать процессор. Также
постепенно растет куча. Затем идет блок futex(0x7ff0c51fd990,
FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 24650, NULL,
FUTEX_BITSET_MATCH_ANY) = 0
sigaltstack({ss_sp=NULL, ss_flags=SS_DISABLE, ss_size=8192}, NULL) = 0
munmap(0x7ff0c5958000, 12288) = 0. Тут futex ждет окончания работы
потоков (по TID). Затем очищается память. И финально exit_group(0)
завершает процесс. Также были захвачены strace каждого из потоков, но там
ничего интересного не происходит - аналогичная инициализация библиотек, а
также

statx(AT_FDCWD, "main.c", AT_STATX_SYNC_AS_STAT, STATX_ALL,
{stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFREG|0644, stx_size=273, ...}) = 0
statx(AT_FDCWD, "main.c",
AT_STATX_SYNC_AS_STAT|AT_SYMLINK_NOFOLLOW, STATX_ALL,
{stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFREG|0644, stx_size=273, ...}) = 0
openat(AT_FDCWD, "main.c", O_RDONLY|O_CLOEXEC) = 3
statx(3, "", AT_STATX_SYNC_AS_STAT|AT_EMPTY_PATH, STATX_ALL,
{stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0,
stx_mode=S_IFREG|0644, stx_size=273, ...}) = 0
mmap(NULL, 273, PROT_READ, MAP_SHARED, 3, 0) = 0x7860f9db5000
write(1, "\33[0m\33[32m2\33[0m:int \33[0m\33[1m\33[31m"..., 46) = 46
write(1, "\n", 1) = 1. Тут программа mmap’ит себе в память содержимое файла, а также выводит в stdout то, что нашла.